DATABASES

DynamoDB Visual Guide

Keys, indexes, capacity, single-table design -- serverless NoSQL at scale.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Partition Keys & Sort Keys

The primary key IS the scaling strategy.

PK only

user_id ->■one item

PK + SK

user_id +■order_date -> many

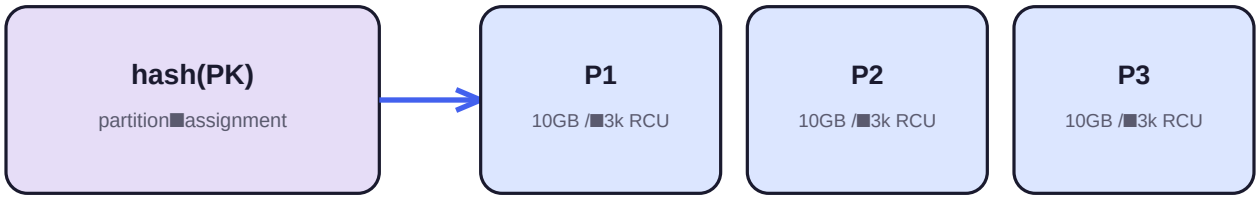
```
-- PK=user:8421, SK=order#2026-10-01  (item collection)
-- PK=user:8421, SK=order#2026-10-02
-- PK=user:8421, SK=profile           (different entity, same user)
-- Query(PK=user:8421) returns ALL of them, sorted by SK
```

Sort keys turn one partition into an ordered collection -- the most powerful DynamoDB pattern.

STEP 2/6

How Partitioning Works

hash(PK) assigns items to partitions -- automatically split as you grow.

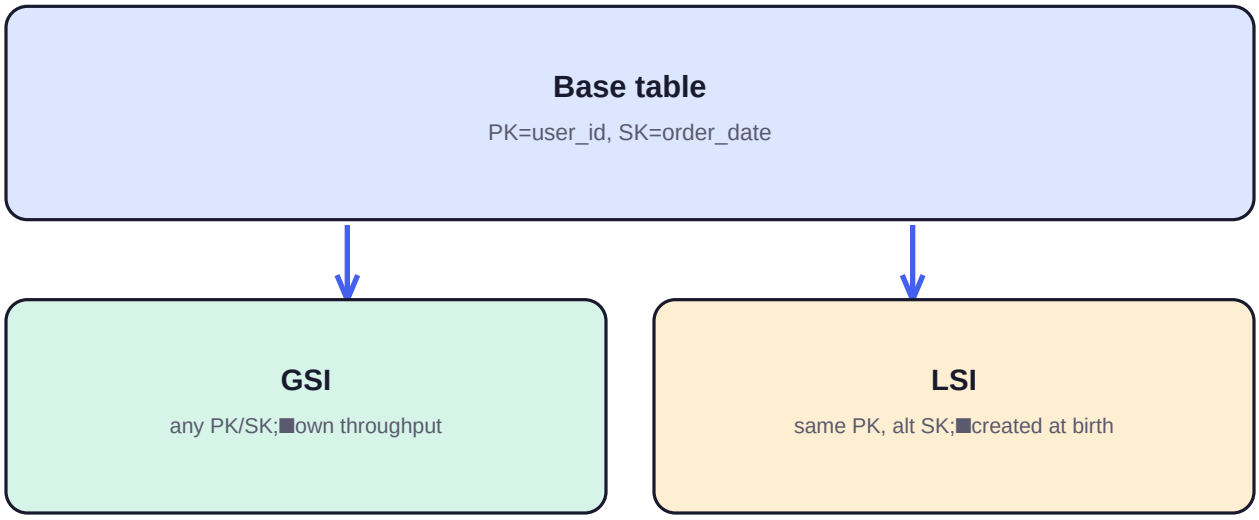


- Partitions split automatically past ~10GB or throughput limits -- no ops work.
- HOT PARTITION: one PK getting all traffic throttles while others idle -- design keys for spread.
- Adaptive capacity helps bursts, but a bad key design is still a bad key design.

STEP 3/6

GSIs & LSIs

Alternate query paths over the same data.



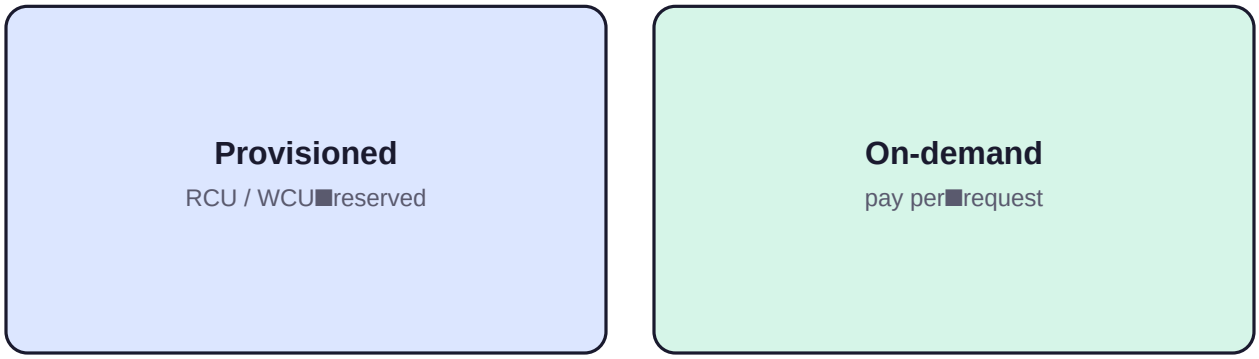
GSI example: PK=status, SK=created_at -- 'all pending orders, newest first'.

GSIs are eventually consistent and cost write throughput. LSIs are strongly consistent but must be defined at table creation.

STEP 4/6

Capacity Modes

Pay for what you provision, or pay per request.



- 1 RCU = 1 strongly-consistent 4KB read/sec. 1 WCU = 1 1KB write/sec.
- Provisioned + auto-scaling wins for steady traffic; on-demand for spiky/unknown.
- Throttling (ProvisionedThroughputExceeded) = raise capacity or fix hot keys.

STEP 5/6

Single-Table Design

One table, many entity types, access patterns first.

```
Table: app  (PK, SK hold EVERYTHING)
  PK=USER#8421   SK=PROFILE#8421      -> user profile
  PK=USER#8421   SK=ORDER#2026-10-01  -> an order
  PK=ORDER#9910  SK=USER#8421         -> inverted lookup (via GSI)
-- each access pattern = one Query on PK (+ optional SK prefix)
```

- List every access pattern BEFORE modeling: 'get user', 'user's orders', 'orders by status'.
- Overloaded keys with prefixes (USER#, ORDER#) let GSIs re-index the same items.
- Trade-off: brilliant reads, but the model is opaque -- document patterns in code.

STEP ✓I✓

Key Takeaways

DynamoDB -- Remember

- The key schema is the database: PK spreads load, SK builds ordered collections.
- Partitions split automatically; hot partitions throttle -- design keys for even spread.
- GSIs add new query paths (eventual, extra cost); LSIs are same-PK alternates fixed at creation.
- Provisioned suits steady traffic, on-demand suits spiky -- know your RCU/WCU math.
- Single-table design: enumerate access patterns first, then overload one table with prefixed keys.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev